

A-MAZE-ING

Algoritma, duvar kodlama, perfect/non-perfect üretim, BFS çözümü ve çıktı formatı için ayrıntılı Türkçe çalışma rehberi

Bu doküman, repodaki güncel **mazegen** kodunu anlayıp savunabilmen için hazırlanmıştır. Kodun ne yaptığını, neden doğru olduğunu ve hangi invariant'larla korunduğunu birlikte ele alır.

Kapsanan dosyalar: model.py, generator.py, solver.py, validation.py, output.py, errors.py ve tests/testall.py

Sürüm tarihi: 12 Eylül 2026

İçindekiler

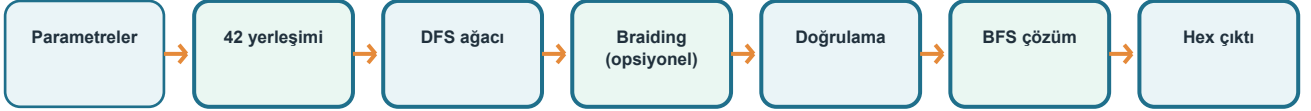
1. Büyük resim: sistem ne yapıyor?	4
2. Duvarların bit düzeyinde gösterimi	5
2.1 Neden dört ayrı boolean yerine tek integer?	5
2.2 Bit işlemleri adım adım	5
2.3 model.py kaynak turu	6
2.4 Maze metodlarının sözleşmesi	8
3. Grafik teorisi: kodun matematiksel temeli	9
3.1 Grid'i grafiğe çevirmek	9
3.2 Neden spanning tree perfect maze demektir?	9
4. MazeGenerator başlangıcı ve girdi doğrulama	10
4.1 Constructor alanları	11
4.2 Her kontrolün nedeni	11
4.3 Determinizm	11
5. Dekoratif 42 deseninin güvenli yerleşimi	12
5.1 Adayların sıralanması	13
5.2 Ön bağlantı testi	13
6. Perfect maze: randomized iterative DFS	14
6.1 Veri yapıları	14
6.2 Döngünün mekanik yürütmesi	14
6.3 Küçük örnek	14
7. Non-perfect maze: braiding ve döngüler	15
7.1 Neden önce tree?	16
7.2 Dead-end azaltma	16
7.3 Minimum loop hedefi	16
8. Tam açık 3x3 alanın engellenmesi	18
8.1 Local pencere sınırları	19
9. generate metodu: orkestrasyon noktası	20
10. BFS ile en kısa çözüm	21
10.1 BFS neden shortest path verir?	21

10.2 Yolun geri kurulması	22
10.3 NESW dönüşümü	22
11. validate_maze: güvenlik ağı	23
11.1 Kontrollerin sırası	24
11.2 Edge sayımında double count önleme	24
11.3 ValidationReport nasıl okunur?	24
12. Hexadecimal çıktı dosyası	25
12.1 serialize_maze neden tekrar doğruluyor?	25
12.2 Exact dosya yapısı	26
13. Hata sınıfları ve hata sınırları	27
14. Testleri okuyarak tasarımı öğrenmek	28
14.1 Her test neyi kanıtlıyor?	30
15. Karmaşıklık ve kaynak kullanımı	31
16. Uçtan uca kullanım ve entegrasyon	32
16.1 Teammate katmanı ne yapmalı?	32
16.2 Modül sınırını bozmama	32
17. Debug rehberi: hata nerede aranır?	33
17.1 İnceleme sırasında faydalı ifadeler	33
18. Savunma soruları ve kısa cevaplar	34
19. Kendini sına	35
19.1 Kod okumadan cevapla	35
19.2 Cevap anahtarı	35
20. Son tekrar: ezberlenmesi gereken çekirdek	36

1. Büyük resim: sistem ne yapıyor?

Bu bölümün görevi, dikdörtgen bir hücre ızgarasını bir grafik olarak ele alıp geçerli bir labirente dönüştürmektir. Her hücre bir düğüm, iki komşu hücre arasındaki açık duvar ise bir kenardır.

Akış, bütün duvarları kapalı bir ızgarayla başlar. Önce güvenli olduğu durumda kapalı F hücreleriyle 42 deseni ayrılır. Sonra DFS bir spanning tree açar. PERFECT=False ise ek duvarlar açılarak döngüler oluşturulur. Sonuç yapısal olarak doğrulanır, BFS ile en kısa yol bulunur ve her hücre tek hexadecimal karaktere çevrilir.



Dosya	Tek sorumluluk
model.py	Hücre, yön, duvar bitleri ve tutarlı duvar işlemleri
generator.py	42 yerleşimi, DFS ağacı ve non-perfect braiding
solver.py	BFS ile en kısa yol ve NESW kodlaması
validation.py	Bağlantı, sınır, simetri, cycle rank ve 3x3 kontrolleri
output.py	Exact hexadecimal metin ve dosyaya yazma
errors.py	Beklenen hata türleri
tests/testall.py	Algoritmik davranış ve format testleri

En önemli tasarım kararı

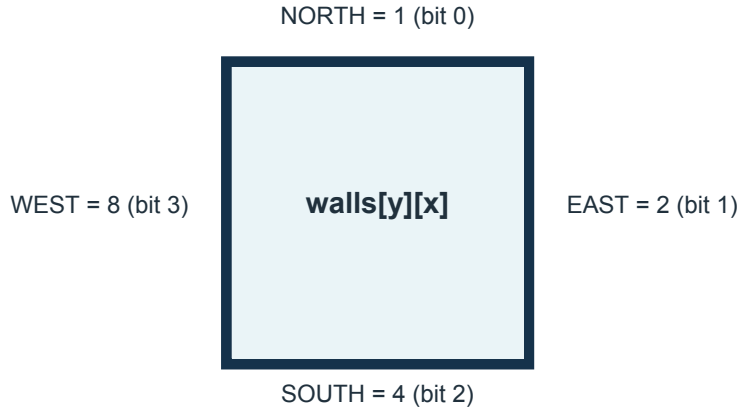
Çözüm yolu maze nesnesinin içine önceden yazılmıyor. Maze yalnızca durumu taşır; solver bu durumdan yol üretir. Aynı şekilde output modülü maze'i değiştirmez, sadece doğrular ve serialize eder.

2. Duvarların bit düzeyinde gösterimi

2.1 Neden dört ayrı boolean yerine tek integer?

Bir hücrenin dört duvarı dört bit ile tutulur. Böylece tüm durum 0 ile 15 arasındaki tek sayıya sığar ve doğrudan bir hexadecimal karaktere dönüşür. Bit set ise duvar kapalı, bit sıfır ise geçit açıktır.

IntFlag seçimi hem integer bit işlemlerini korur hem de NORTH gibi okunabilir isimler verir. Coordinate alias'ı da bütün modüllerde (x, y) sözleşmesini görünür kılar.



Yön	Değer	Binary	Anlam
NORTH	1	0001	Kuzey duvarı kapalı
EAST	2	0010	Doğu duvarı kapalı
SOUTH	4	0100	Güney duvarı kapalı
WEST	8	1000	Batı duvarı kapalı
ALL_WALLS	15 = F	1111	Dört duvar da kapalı

2.2 Bit işlemleri adım adım

- Kontrol: **value & direction**. Sonuç sıfır değilse ilgili duvar kapalıdır.
- Açma: **value &= ~direction**. Ters maske yalnız hedef biti sıfırlar.
- Kapama: **value |= direction**. OR yalnız hedef biti bire çeker.
- Hex dönüşümü: **format(value, 'X')**. 10-15 arası A-F olur.

```
# Örnek: hücre D = 1101 = NORTH + SOUTH + WEST
value = 0xD
east_closed = bool(value & Direction.EAST) # False
value |= int(Direction.EAST)               # F: doğuyu kapat
value &= ~int(Direction.NORTH)             # E: kuzeyi aç
```

D7 örneği

Yatay iki hücre arasında tek geçit açılırsa sol hücre D olur (doğu açık), sağ hücre 7 olur (batı açık). Bu nedenle 2x1 örnek çıktının ilk satırı tam olarak D7'dir.

2.3 model.py kaynak turu

```

9 | Coordinate = tuple[int, int]
10 |
11 |
12 | class Direction(IntFlag):
13 |     """Wall bits used by the required hexadecimal representation."""
14 |
15 |     NORTH = 1
16 |     EAST = 2
17 |     SOUTH = 4
18 |     WEST = 8
19 |
20 |     @property
21 |     def opposite(self) -> "Direction":
22 |         """Return the same shared wall as seen from the next cell."""
23 |         return OPPOSITES[self]
24 |
25 |     @property
26 |     def offset(self) -> Coordinate:
27 |         """Return the x/y movement associated with this direction."""
28 |         return OFFSETS[self]
29 |
30 |     @property
31 |     def letter(self) -> str:
32 |         """Return the direction's one-letter solution encoding."""
33 |         return LETTERS[self]
34 |
35 |
36 | OPPOSITES: dict[Direction, Direction] = {
37 |     Direction.NORTH: Direction.SOUTH,
38 |     Direction.EAST: Direction.WEST,
39 |     Direction.SOUTH: Direction.NORTH,
40 |     Direction.WEST: Direction.EAST,
41 | }
42 | OFFSETS: dict[Direction, Coordinate] = {
43 |     Direction.NORTH: (0, -1),
44 |     Direction.EAST: (1, 0),
45 |     Direction.SOUTH: (0, 1),
46 |     Direction.WEST: (-1, 0),
47 | }
48 | LETTERS: dict[Direction, str] = {
49 |     Direction.NORTH: "N",
50 |     Direction.EAST: "E",
51 |     Direction.SOUTH: "S",
52 |     Direction.WEST: "W",
53 | }
54 | ALL_WALLS = 0xF

```

OPPOSITES sözlüğü shared-wall coherence için kritiktir. Bir hücrenin EAST duvarı, doğu komşusunun WEST duvarıyla aynı fiziksel duvardır. OFFSETS hareketi, LETTERS ise çözüm çıktısını merkezileştirir.

walls dizisinin indeks sırası **walls[y][x]**'tir. Koordinatın ise **(x, y)** olması en sık karıştırılan noktadır.

```

57 | @dataclass
58 | class Maze:
59 |     """A generated rectangular maze.
60 |
61 |     Coordinates are always `(x, y)`. Each wall value uses bits N=1, E=2,
62 |     S=4, and W=8; a set bit means the wall is closed.
63 |     """
64 |
65 |     width: int
66 |     height: int
67 |     entry: Coordinate
68 |     exit: Coordinate
69 |     seed: int
70 |     perfect: bool
71 |     walls: list[list[int]]
72 |     blocked: frozenset[Coordinate]
73 |
74 |     def in_bounds(self, cell: Coordinate) -> bool:
75 |         """Return whether a cell lies inside the grid."""
76 |         x_pos, y_pos = cell
77 |         return 0 <= x_pos < self.width and 0 <= y_pos < self.height
78 |
79 |     def traversable_cells(self) -> Iterator[Coordinate]:
80 |         """Yield cells that are not closed decorative cells."""
81 |         for y_pos in range(self.height):
82 |             for x_pos in range(self.width):
83 |                 cell = (x_pos, y_pos)
84 |                 if cell not in self.blocked:

```

```

85 |         yield cell
86 |
87 |     def neighbors(
88 |         self, cell: Coordinate
89 |     ) -> Iterator[tuple[Coordinate, Direction]]:
90 |         """Yield each in-bounds orthogonal neighbor and its direction."""
91 |         x_pos, y_pos = cell
92 |         for direction in Direction:
93 |             dx, dy = direction.offset
94 |             neighbor = (x_pos + dx, y_pos + dy)
95 |             if self.in_bounds(neighbor):
96 |                 yield neighbor, direction
97 |
98 |     def has_wall(self, cell: Coordinate, direction: Direction) -> bool:
99 |         """Return whether one wall of a cell is closed."""
100 |         if not self.in_bounds(cell):
101 |             raise ValueError(f"cell is outside the maze: {cell}")
102 |         x_pos, y_pos = cell
103 |         return bool(self.walls[y_pos][x_pos] & direction)
104 |
105 |     def remove_wall(self, cell: Coordinate, direction: Direction) -> None:
106 |         """Open a shared wall on both adjacent cells."""
107 |         if not self.in_bounds(cell):
108 |             raise ValueError(f"cell is outside the maze: {cell}")
109 |         dx, dy = direction.offset
110 |         neighbor = (cell[0] + dx, cell[1] + dy)
111 |         if not self.in_bounds(neighbor):
112 |             raise ValueError("cannot open an external border")
113 |         if cell in self.blocked or neighbor in self.blocked:
114 |             raise ValueError("cannot open a decorative closed cell")
115 |         self.walls[cell[1]][cell[0]] &= ~int(direction)
116 |         self.walls[neighbor[1]][neighbor[0]] &= ~int(direction.opposite)
117 |
118 |     def add_wall(self, cell: Coordinate, direction: Direction) -> None:
119 |         """Close a shared wall on both adjacent cells."""
120 |         if not self.in_bounds(cell):
121 |             raise ValueError(f"cell is outside the maze: {cell}")
122 |         dx, dy = direction.offset
123 |         neighbor = (cell[0] + dx, cell[1] + dy)
124 |         if not self.in_bounds(neighbor):
125 |             raise ValueError("cannot address an external shared wall")
126 |         self.walls[cell[1]][cell[0]] |= int(direction)
127 |         self.walls[neighbor[1]][neighbor[0]] |= int(direction.opposite)
128 |
129 |     def open_neighbors(self, cell: Coordinate) -> Iterator[Coordinate]:
130 |         """Yield traversable neighbors joined by an open passage."""
131 |         for neighbor, direction in self.neighbors(cell):
132 |             if neighbor not in self.blocked and not self.has_wall(
133 |                 cell, direction
134 |             ):
135 |                 yield neighbor
136 |
137 |     def degree(self, cell: Coordinate) -> int:
138 |         """Return the number of open passages incident to a cell."""
139 |         return sum(1 for _neighbor in self.open_neighbors(cell))
140 |
141 |     def to_hex_lines(self) -> list[str]:
142 |         """Return the maze grid as uppercase hexadecimal rows."""
143 |         return [
144 |             "".join(format(value, "X") for value in row)
145 |             for row in self.walls
146 |         ]

```

2.4 Maze metodlarının sözleşmesi

Metot	Girdi	Garanti / sonuç
in_bounds	(x, y)	Koordinatın grid içinde olup olmadığını döndürür
traversable_cells	-	blocked olmayan hücreleri row-major sırada üretir
neighbors	cell	Sınır içindeki komşu ve yön çiftlerini üretir
has_wall	cell, direction	Bit maskesiyle duvarı sorgular
remove_wall	cell, direction	İki taraftaki karşılıklı bitleri birlikte temizler
add_wall	cell, direction	İki taraftaki karşılıklı bitleri birlikte set eder
open_neighbors	cell	Duvarsız ve blocked olmayan komşuları verir
degree	cell	Açık komşu sayısını verir
to_hex_lines	-	Her satırı uppercase hex string yapar

3. Grafik teorisi: kodun matematiksel temeli

3.1 Grid'i grafiğe çevirmek

Traversable her hücre bir vertex V 'dir. Aralarında açık shared wall bulunan iki ortogonal komşu bir edge E oluşturur. 42 hücreleri graph dışındadır; çünkü dört duvarı kapalı dekoratif engellerdir.

Bağlı bir grafikte cycle rank formülü $E - V + 1$ 'dir. Sonuç 0 ise döngü yoktur. 1 ise tek bağımsız döngü, 2 veya daha büyükse birden fazla alternatif rota vardır.

Durum	Bağlılık	Kenar sayısı	Cycle rank	Yorum
Perfect	Bağlı	$E = V - 1$	0	Ağaç; her çift arasında tek yol
Tek ekstra açıklık	Bağlı	$E = V$	1	Yetersiz non-perfect
Non-perfect	Bağlı	$E \geq V + 1$	≥ 2	Birden fazla bağımsız döngü

3.2 Neden spanning tree perfect maze demektir?

DFS yalnız daha önce ziyaret edilmemiş bir komşuya geçerken duvar açar. İlk hücre dışındaki her vertex tam bir kez mevcut yapıya bağlanır. V vertex için açılan kenar sayısı $V - 1$ olur.

DFS bütün traversable hücrelere ulaştığı için grafik bağlıdır. Bağlı ve $V - 1$ kenarlı bir undirected graph bir ağaçtır. Ağaçta iki vertex arasında iki ayrı yol olsaydı bu iki yol birlikte cycle oluştururdu; bu çelişki nedeniyle yol tektir.

Savunmada tek cümlelik ispat

Her yeni hücreyi mevcut yapıya tam bir kenarla bağlıyorum; bütün hücreler ulaşıldığında bağlı V düğümlü, $V-1$ kenarlı bir graph elde ediyorum. Bu graph tanım gereği bir tree'dir.

4. MazeGenerator başlangıcı ve girdi doğrulama

```

18 | PATTERN: tuple[str, ...] = (
19 |     "X.X.XXX",
20 |     "X.X...X",
21 |     "XXX.XXX",
22 |     "..X.X..",
23 |     "..X.XXX",
24 | )
25 | PATTERN_WIDTH = len(PATTERN[0])
26 | PATTERN_HEIGHT = len(PATTERN)
27 | MAX_CELLS = 250_000
28 |
29 |
30 | class MazeGenerator:
31 |     """Generate deterministic perfect or non-perfect mazes."""
32 |
33 |     def __init__(
34 |         self,
35 |         width: int,
36 |         height: int,
37 |         entry: Coordinate,
38 |         exit: Coordinate,
39 |         seed: int | None = None,
40 |         perfect: bool = True,
41 |         include_pattern: bool = True,
42 |     ) -> None:
43 |         """Store and validate generation parameters."""
44 |         self.width = width
45 |         self.height = height
46 |         self.entry = entry
47 |         self.exit = exit
48 |         self.seed = secrets.randbits(64) if seed is None else seed
49 |         self.perfect = perfect
50 |         self.include_pattern = include_pattern
51 |         self.maze: Maze | None = None
52 |         self.report: ValidationReport | None = None
53 |         self.diagnostics: list[str] = []
54 |         self._validate_arguments()
55 |
56 |     def _validate_arguments(self) -> None:
57 |         """Reject invalid coordinates, modes, and impossible dimensions."""
58 |         if isinstance(self.width, bool) or not isinstance(self.width, int):
59 |             raise MazeGenerationError("width must be an integer")
60 |         if isinstance(self.height, bool) or not isinstance(self.height, int):
61 |             raise MazeGenerationError("height must be an integer")
62 |         if self.width <= 0 or self.height <= 0:
63 |             raise MazeGenerationError("width and height must be positive")
64 |         if self.width * self.height > MAX_CELLS:
65 |             raise MazeGenerationError(
66 |                 f"maze cannot exceed {MAX_CELLS} cells"
67 |             )
68 |         if self.width * self.height < 2:
69 |             raise MazeGenerationError("maze needs at least two cells")
70 |         for label, cell in (("entry", self.entry), ("exit", self.exit)):
71 |             if not isinstance(cell, tuple) or len(cell) != 2:
72 |                 raise MazeGenerationError(f"{label} must be an x,y pair")
73 |             if any(
74 |                 isinstance(value, bool) or not isinstance(value, int)
75 |                 for value in cell
76 |             ):
77 |                 raise MazeGenerationError(
78 |                     f"{label} coordinates must be integers"
79 |                 )
80 |             if not self._in_bounds(cell):
81 |                 raise MazeGenerationError(f"{label} is outside the maze")
82 |         if self.entry == self.exit:
83 |             raise MazeGenerationError("entry and exit must be different")
84 |         if not isinstance(self.seed, int) or isinstance(self.seed, bool):
85 |             raise MazeGenerationError("seed must be an integer")
86 |         if not isinstance(self.perfect, bool):
87 |             raise MazeGenerationError("perfect must be a boolean")
88 |         if not isinstance(self.include_pattern, bool):
89 |             raise MazeGenerationError("include_pattern must be a boolean")
90 |         if not self.perfect and self._maximum_cycle_rank(set()) < 2:
91 |             raise MazeGenerationError(
92 |                 "dimensions cannot contain two independent cycles"
93 |             )
94 |
95 |     def _in_bounds(self, cell: Coordinate) -> bool:
96 |         """Return whether a cell lies inside the requested grid."""

```

```

97 |         return (
98 |             0 <= cell[0] < self.width
99 |             and 0 <= cell[1] < self.height
100 |         )

```

4.1 Constructor alanları

- width / height: grid boyutları.
- entry / exit: (x, y) başlangıç ve hedef hücreleri.
- seed: verilmezse secrets.randbits(64) ile seçilir; verilirse local random.Random kullanır.
- perfect: True ise tree korunur, False ise braiding yapılır.
- include_pattern: 42 deseninin denenip denenmeyeceğini belirler.
- maze ve report: generate çağrısından sonra son sonucu ve istatistikleri taşır.
- diagnostics: pattern neden atlandı gibi fatal olmayan mesajları taşır.

4.2 Her kontrolün nedeni

Kontrol	Önlediği hata
bool ayrıca reddediliyor	Python'da bool, int alt sınıfıdır; True genişlik kabul edilmesin
width, height > 0	Boş veya negatif grid
MAX_CELLS	Kontrolsüz bellek ve çalışma süresi
En az iki hücre	Entry ve exit farklı olamazsa geçerli maze oluşmaz
Tuple uzunluğu 2	Koordinat formatı bozukluğu
in_bounds	Negatif veya taşan entry/exit
entry != exit	Sıfır uzunluklu anlamsız problem
non-perfect maximum rank	Geometrik olarak iki cycle imkansızsa erken hata

4.3 Determinizm

Global random state kullanılmaz. generate içinde yalnız random.Random(self.seed) oluşturulur. Aynı boyutlar, aynı giriş/çıkış, aynı mode ve aynı seed aynı seçim sırasını ve dolayısıyla aynı wall grid'ini üretir.

Seed None ise secrets modülü 64-bit bir değer seçer. Seçilen değer self.seed içinde kalır; dış katman bunu göstererek maze'i yeniden ürettirebilir.

5. Dekoratif 42 deseninin güvenli yerleşimi

```
X.X.XXX
X.X...X
XXX.XXX
..X.X..
..X.XXX
```

Desen 7x5 bounding box içinde 20 adet X hücresi içerir. Yalnız X konumları blocked set'ine girer; nokta konumları normal koridordur. Blocked hücreler başlangıçta F'tir ve onlara hiçbir duvar açılmaz.

En az bir hücrelik güvenlik halkası için width ≥ 9 ve height ≥ 7 aranır. Daha küçük maze'de generator hata vermez; deseni atlar ve diagnostics içine açıklama koyar.

```
114 |     def _maximum_cycle_rank(self, blocked: set[Coordinate]) -> int:
115 |         """Return the cycle rank of the fully open available grid."""
116 |         vertices = self.width * self.height - len(blocked)
117 |         edges = 0
118 |         for y_pos in range(self.height):
119 |             for x_pos in range(self.width):
120 |                 cell = (x_pos, y_pos)
121 |                 if cell in blocked:
122 |                     continue
123 |                 if x_pos + 1 < self.width:
124 |                     edges += (x_pos + 1, y_pos) not in blocked
125 |                 if y_pos + 1 < self.height:
126 |                     edges += (x_pos, y_pos + 1) not in blocked
127 |         return edges - vertices + 1
128 |
129 |     def _cells_are_connected(self, blocked: set[Coordinate]) -> bool:
130 |         """Check connectivity before accepting a decorative pattern."""
131 |         cells = {
132 |             (x_pos, y_pos)
133 |             for y_pos in range(self.height)
134 |             for x_pos in range(self.width)
135 |             if (x_pos, y_pos) not in blocked
136 |         }
137 |         if not cells:
138 |             return False
139 |         reached: set[Coordinate] = set()
140 |         pending = [next(iter(cells))]
141 |         while pending:
142 |             current = pending.pop()
143 |             if current in reached:
144 |                 continue
145 |             reached.add(current)
146 |             pending.extend(
147 |                 neighbor
148 |                 for neighbor, _direction in self._grid_neighbors(
149 |                     current, blocked
150 |                 )
151 |                 if neighbor not in reached
152 |             )
153 |         return reached == cells
154 |
155 |     @staticmethod
156 |     def _pattern_at(left: int, top: int) -> set[Coordinate]:
157 |         """Return the X coordinates of a translated 42 pattern."""
158 |         return {
159 |             (left + x_pos, top + y_pos)
160 |             for y_pos, row in enumerate(PATTERN)
161 |             for x_pos, marker in enumerate(row)
162 |             if marker == "X"
163 |         }
164 |
165 |     def _choose_pattern(self) -> set[Coordinate]:
166 |         """Find a safe pattern placement nearest the maze center."""
167 |         if not self.include_pattern:
168 |             return set()
169 |         if self.width < PATTERN_WIDTH + 2 or self.height < PATTERN_HEIGHT + 2:
170 |             self.diagnostics.append(
171 |                 "42 pattern omitted: maze is too small for a safe margin"
172 |             )
173 |             return set()
174 |         preferred = (
175 |             (self.width - PATTERN_WIDTH) // 2,
176 |             (self.height - PATTERN_HEIGHT) // 2,
177 |         )
178 |         origins = [
```

```
179 |         (left, top)
180 |         for top in range(1, self.height - PATTERN_HEIGHT)
181 |         for left in range(1, self.width - PATTERN_WIDTH)
182 |     ]
183 |     origins.sort(
184 |         key=lambda origin: (
185 |             abs(origin[0] - preferred[0])
186 |             + abs(origin[1] - preferred[1]),
187 |             origin[1],
188 |             origin[0],
189 |         )
190 |     )
191 |     protected = {
192 |         self.entry,
193 |         self.exit,
194 |         (0, 0),
195 |         (self.width - 1, 0),
196 |         (0, self.height - 1),
197 |         (self.width - 1, self.height - 1),
198 |         center_cell(self.width, self.height),
199 |     }
200 |     for left, top in origins:
201 |         blocked = self._pattern_at(left, top)
202 |         if blocked & protected or not self._cells_are_connected(blocked):
203 |             continue
204 |         if not self.perfect and self._maximum_cycle_rank(blocked) < 2:
205 |             continue
206 |         return blocked
207 |     self.diagnostics.append(
208 |         "42 pattern omitted: no safe connected placement was found"
209 |     )
210 |     return set()
```

5.1 Adayların sıralanması

Bütün güvenli origin konumları hazırlanır. Manhattan distance ile geometrik merkeze en yakın aday önce denenir. Eşitlik durumunda y, sonra x sırası deterministik tie-breaker'dır.

Aday; entry, exit, dört köşe veya seçilen center hücrelerinden biriyle çakışırsa reddedilir. Kalan grid connected değilse veya non-perfect için iki cycle kapasitesi kalmıyorsa da reddedilir.

5.2 Ön bağlantı testi

`_cells_are_connected` gerçek duvarlara bakmaz; blocked hücreler çıkarılmış teorik grid üzerinde DFS yapar. Amaç desen yerleşiminin maze daha üretilmeden koridor alanını iki parçaya bölüp bölmediğini görmektir.

Even dimension center convention: `center_cell(width, height) = (width // 2, height // 2)`. Yani çift ölçüde orta dörtlünün sağ-alt hücresi seçilir.

6. Perfect maze: randomized iterative DFS

```

212 |     def _carve_tree(self, maze: Maze, rng: random.Random) -> None:
213 |         """Carve a randomized depth-first spanning tree."""
214 |         cells = list(maze.traversable_cells())
215 |         start = rng.choice(cells)
216 |         reached = {start}
217 |         stack = [start]
218 |         blocked = set(maze.blocked)
219 |         while stack:
220 |             current = stack[-1]
221 |             candidates = [
222 |                 item
223 |                 for item in self._grid_neighbors(current, blocked)
224 |                 if item[0] not in reached
225 |             ]
226 |             if not candidates:
227 |                 stack.pop()
228 |                 continue
229 |             neighbor, direction = rng.choice(candidates)
230 |             maze.remove_wall(current, direction)
231 |             reached.add(neighbor)
232 |             stack.append(neighbor)
233 |             if len(reached) != len(cells):
234 |                 raise MazeGenerationError("could not connect every corridor cell")

```

6.1 Veri yapıları

- cells: blocked olmayan bütün vertex'ler.
- start: seed'li RNG ile seçilen başlangıç vertex'i.
- reached: daha önce tree'ye eklenmiş hücreler; tekrar edge açılmasını engeller.
- stack: aktif DFS yolunu tutar; Python recursion limitine bağımlılık yoktur.

6.2 Döngünün mekanik yürütmesi

Adım	İşlem	Sonuç
1	stack[-1] ile current seç	Aktif yolun ucunda kal
2	Ulaşılmamış komşuları filtrele	Cycle açabilecek eski vertex'leri çıkar
3a	Candidate yoksa pop	Backtracking yap
3b	Candidate varsa random choice	Seed'e bağlı çeşitlilik sağla
4	Shared wall'u iki taraftan aç	Graph'a bir edge ekle
5	Neighbor'u reached ve stack'e ekle	DFS daha derine ilerlesin
6	Stack boşalana kadar sürdür	Bütün component tamamlanır

6.3 Küçük örnek

```

Başlangıç: stack=[A], reached={A}
A -> B aç: stack=[A,B], reached={A,B}
B -> C aç: stack=[A,B,C], reached={A,B,C}
C çıkmaz: stack=[A,B]          # pop ile geri dön
B -> D aç: stack=[A,B,D], reached={A,B,C,D}
... stack boşaldığında bütün hücreler tree içindedir

```

Neden recursive değil?

20x15 küçük olsa da büyük bir DFS yolu Python recursion limitini aşabilir. Açık stack aynı algoritmayı güvenli ve gözlemlenebilir biçimde uygular.

7. Non-perfect maze: braiding ve döngüler

```

236 |     @staticmethod
237 |     def _closed_edges(
238 |         maze: Maze,
239 |     ) -> list[tuple[Coordinate, Direction]]:
240 |         """Return each closed shared wall exactly once."""
241 |         edges: list[tuple[Coordinate, Direction]] = []
242 |         for cell in maze.traversable_cells():
243 |             for direction in (Direction.EAST, Direction.SOUTH):
244 |                 dx, dy = direction.offset
245 |                 neighbor = (cell[0] + dx, cell[1] + dy)
246 |                 if (
247 |                     maze.in_bounds(neighbor)
248 |                     and neighbor not in maze.blocked
249 |                     and maze.has_wall(cell, direction)
250 |                 ):
251 |                     edges.append((cell, direction))
252 |         return edges
253 |
254 |     def _opening_creates_open_3x3(
255 |         self, maze: Maze, cell: Coordinate, direction: Direction
256 |     ) -> bool:
257 |         """Test affected 3-by-3 windows before opening a wall."""
258 |         dx, dy = direction.offset
259 |         neighbor = (cell[0] + dx, cell[1] + dy)
260 |         maze.remove_wall(cell, direction)
261 |         min_x = min(cell[0], neighbor[0])
262 |         max_x = max(cell[0], neighbor[0])
263 |         min_y = min(cell[1], neighbor[1])
264 |         max_y = max(cell[1], neighbor[1])
265 |         creates_open_area = any(
266 |             window_is_open(maze, left, top)
267 |             for top in range(
268 |                 max(0, max_y - 2),
269 |                 min(min_y, maze.height - 3) + 1,
270 |             )
271 |             for left in range(
272 |                 max(0, max_x - 2),
273 |                 min(min_x, maze.width - 3) + 1,
274 |             )
275 |         )
276 |         maze.add_wall(cell, direction)
277 |         return creates_open_area
278 |
279 |     def _try_open(
280 |         self, maze: Maze, cell: Coordinate, direction: Direction
281 |     ) -> bool:
282 |         """Open a wall unless it violates the 3-by-3 invariant."""
283 |         if not maze.has_wall(cell, direction):
284 |             return False
285 |         if self._opening_creates_open_3x3(maze, cell, direction):
286 |             return False
287 |         maze.remove_wall(cell, direction)
288 |         return True
289 |
290 |     def _braid(self, maze: Maze, rng: random.Random) -> None:
291 |         """Reduce dead ends and create several independent cycles."""
292 |         opened = 0
293 |         for _pass_number in range(3):
294 |             dead_ends = [
295 |                 cell
296 |                 for cell in maze.traversable_cells()
297 |                 if maze.degree(cell) == 1
298 |             ]
299 |             rng.shuffle(dead_ends)
300 |             changed = False
301 |             for cell in dead_ends:
302 |                 if maze.degree(cell) != 1:
303 |                     continue
304 |                 candidates = [
305 |                     (neighbor, direction)
306 |                     for neighbor, direction in maze.neighbors(cell)
307 |                     if neighbor not in maze.blocked
308 |                     and maze.has_wall(cell, direction)
309 |                 ]
310 |                 rng.shuffle(candidates)
311 |                 candidates.sort(key=lambda item: maze.degree(item[0]))
312 |                 for _neighbor, direction in candidates:
313 |                     if self._try_open(maze, cell, direction):
314 |                         opened += 1

```

```

315 |                 changed = True
316 |                 break
317 |             if not changed:
318 |                 break
319 |
320 |             target_rank = max(
321 |                 2, sum(1 for _cell in maze.traversable_cells()) // 12
322 |             )
323 |             remaining = self._closed_edges(maze)
324 |             rng.shuffle(remaining)
325 |             for cell, direction in remaining:
326 |                 if opened >= target_rank:
327 |                     break
328 |                 if self._try_open(maze, cell, direction):
329 |                     opened += 1
330 |
331 |     def generate(self) -> Maze:
332 |         """Generate and structurally validate one maze."""
333 |         self.diagnostics.clear()
334 |         blocked = self._choose_pattern()
335 |         maze = Maze(
336 |             width=self.width,
337 |             height=self.height,
338 |             entry=self.entry,
339 |             exit=self.exit,
340 |             seed=self.seed,
341 |             perfect=self.perfect,
342 |             walls=[
343 |                 [ALL_WALLS for _x_pos in range(self.width)]
344 |                 for _y_pos in range(self.height)
345 |             ],
346 |             blocked=frozenset(blocked),
347 |         )
348 |         rng = random.Random(self.seed)
349 |         self._carve_tree(maze, rng)
350 |         if not self.perfect:
351 |             self._braid(maze, rng)
352 |         self.report = validate_maze(maze)
353 |         self.maze = maze
354 |         return maze
355 |
356 |     def shortest_path(self) -> list[Coordinate]:
357 |         """Return a shortest solution for the last generated maze."""
358 |         if self.maze is None:
359 |             raise MazeGenerationError("generate the maze before solving it")
360 |         return shortest_path(self.maze)

```

7.1 Neden önce tree?

Tree ile başlamak bağlantıyı bedavaya garanti eder. Daha sonra kapalı bir internal wall açmak hiçbir hücreyi koparmaz; yalnız iki zaten ulaşılabilir vertex arasında ek edge oluşturur.

Connected tree'nin cycle rank'ı 0'dır. Tree içinde kapalı bir duvarı açmak E'yi 1 artırır, V ve component sayısını değiştirmez. Dolayısıyla her başarılı ek açıklık cycle rank'ı tam 1 artırır. opened sayacının matematiksel anlamı budur.

7.2 Dead-end azaltma

- $\text{degree}(\text{cell}) == 1$ ise hücrenin yalnız bir çıkışı vardır.
- Dead-end listesi seed'li RNG ile karıştırılır; aynı seed yine deterministiktir.
- Zaten açık olmayan, blocked olmayan komşular candidate olur.
- Candidate'lar komşu degree'sine göre sıralanır; düşük degree'li bölgeler önce bağlanır.
- Üç pass, ilk açıklıkların başka dead-end'leri değiştirmesine fırsat verir.

7.3 Minimum loop hedefi

$\text{target_rank} = \max(2, \text{traversable_count} // 12)$. Zorunlu alt sınır iki bağımsız cycle'dır. Büyük maze'lerde yalnız iki loop ile yetinmek yerine hücre sayısıyla büyüyen daha oyun-benzeri bir ağ oluşturulur.

Dead-end pass hedefi zaten aşmış olabilir. Aşmadıysa `_closed_edges` yalnız EAST ve SOUTH yönlerine bakarak her shared wall'u bir kere listeler; liste karıştırılır ve hedefe kadar güvenli duvarlar açılır.

Seed 42, 20x15	Perfect	Non-perfect
Traversable vertex	280	280
Open edge	279	306
Cycle rank	0	27
Shortest route	67 adım	45 adım
Analyzer ordinary dead-end	Perfect modda sınırlanmaz	0

8. Tam açık 3x3 alanın engellenmesi

Bir 3x3 blokta dokuz traversable hücre bulunur. Tam açık sayılması için üç satırdaki altı yatay internal edge ve üç sütundaki altı dikey internal edge açık olmalıdır. Toplam 12 internal duvar kontrol edilir.

Perfect tree zaten cycle içermediği için tamamen açık 3x3 oluşturamaz. Risk braiding sırasında doğar. Bu nedenle her candidate edge önce geçici açılır, yalnız o edge'i içerebilecek local windows incelenir ve duvar geri kapatılır.

Güvenliyse `_try_open` kalıcı olarak açar.

```

28 | def _count_edges(maze: Maze) -> int:
29 |     """Count each undirected open passage once."""
30 |     edges = 0
31 |     for cell in maze.traversable_cells():
32 |         for direction in (Direction.EAST, Direction.SOUTH):
33 |             dx, dy = direction.offset
34 |             neighbor = (cell[0] + dx, cell[1] + dy)
35 |             if maze.in_bounds(neighbor) and neighbor not in maze.blocked:
36 |                 edges += not maze.has_wall(cell, direction)
37 |     return edges
38 |
39 |
40 | def _window_is_open(maze: Maze, left: int, top: int) -> bool:
41 |     """Return whether one 3-by-3 window has all internal walls open."""
42 |     cells = {
43 |         (x_pos, y_pos)
44 |         for y_pos in range(top, top + 3)
45 |         for x_pos in range(left, left + 3)
46 |     }
47 |     if cells & maze.blocked:
48 |         return False
49 |     horizontal = all(
50 |         not maze.has_wall((x_pos, y_pos), Direction.EAST)
51 |         for y_pos in range(top, top + 3)
52 |         for x_pos in range(left, left + 2)
53 |     )
54 |     vertical = all(
55 |         not maze.has_wall((x_pos, y_pos), Direction.SOUTH)
56 |         for y_pos in range(top, top + 2)
57 |         for x_pos in range(left, left + 3)
58 |     )
59 |     return horizontal and vertical
60 |
61 |
62 | def has_open_3x3(maze: Maze) -> bool:
63 |     """Return whether the maze contains a fully open 3-by-3 area."""
64 |     return any(
65 |         _window_is_open(maze, left, top)
66 |         for top in range(maze.height - 2)
67 |         for left in range(maze.width - 2)
68 |     )
69 |
70 |
71 | def _opening_creates_open_3x3(
72 |     self, maze: Maze, cell: Coordinate, direction: Direction
73 | ) -> bool:
74 |     """Test affected 3-by-3 windows before opening a wall."""
75 |     dx, dy = direction.offset
76 |     neighbor = (cell[0] + dx, cell[1] + dy)
77 |     maze.remove_wall(cell, direction)
78 |     min_x = min(cell[0], neighbor[0])
79 |     max_x = max(cell[0], neighbor[0])
80 |     min_y = min(cell[1], neighbor[1])
81 |     max_y = max(cell[1], neighbor[1])
82 |     creates_open_area = any(
83 |         window_is_open(maze, left, top)
84 |         for top in range(
85 |             max(0, max_y - 2),
86 |             min(min_y, maze.height - 3) + 1,
87 |         )
88 |         for left in range(
89 |             max(0, max_x - 2),
90 |             min(min_x, maze.width - 3) + 1,
91 |         )
92 |     )
93 |     maze.add_wall(cell, direction)
94 |     return creates_open_area
95 |
96 | def _try_open(
97 |     self, maze: Maze, cell: Coordinate, direction: Direction
98 | ) -> bool:

```

```
282 |         """Open a wall unless it violates the 3-by-3 invariant."""
283 |         if not maze.has_wall(cell, direction):
284 |             return False
285 |         if self._opening_creates_open_3x3(maze, cell, direction):
286 |             return False
287 |         maze.remove_wall(cell, direction)
288 |         return True
```

8.1 Local pencere sınırları

Değişen edge'in iki ucunun min/max x ve y değerleri alınır. 3x3 pencerenin sol üst köşesi edge'in en fazla iki hücre solunda veya üstünde olabilir. range sınırları aynı zamanda maze dışını kırpar.

Bu local kontrol, her candidate için bütün W*H pencereleri dolaşmak yerine sabit sayıda pencere kontrol eder. Final validation yine tüm grid'i tarar; optimizasyon doğruluk kontrolünün yerini almaz.

9. generate metodu: orkestrasyon noktası

```

341 |         perfect=self.perfect,
342 |         walls=[
343 |             [ALL_WALLS for _x_pos in range(self.width)]
344 |             for _y_pos in range(self.height)
345 |         ],
346 |         blocked=frozenset(blocked),
347 |     )
348 |     rng = random.Random(self.seed)
349 |     self._carve_tree(maze, rng)
350 |     if not self.perfect:
351 |         self._braid(maze, rng)
352 |     self.report = validate_maze(maze)
353 |     self.maze = maze
354 |     return maze
355 |
356 |     def shortest_path(self) -> list[Coordinate]:
357 |         """Return a shortest solution for the last generated maze."""
358 |         if self.maze is None:
359 |             raise MazeGenerationError("generate the maze before solving it")
360 |         return shortest_path(self.maze)

```

Sıra	İşlem	Değişen durum
1	diagnostics.clear	Önceki generation mesajları silinir
2	_choose_pattern	blocked set belirlenir
3	Maze oluşturma	Bütün walls hücreleri F başlar
4	random.Random(seed)	Lokal deterministik RNG hazırlanır
5	_carve_tree	Connected spanning tree açılır
6	_braid	Yalnız non-perfect modda extra edges açılır
7	validate_maze	Bağımsız invariant kontrolü yapılır
8	maze/report sakla	API kullanıcısı sonuca erişir

Başlangıçta neden her şey F?

Kapalıdan açığa doğru üretim dış sınırları korumayı kolaylaştırır. Algoritma yalnız in-bounds iki normal hücre arasındaki duvarları açabildiği için external border yanlışlıkla açılmaz.

10. BFS ile en kısa çözüm

```

1 | """Shortest-path solving kept separate from maze generation."""
2 |
3 | from __future__ import annotations
4 |
5 | from collections import deque
6 |
7 | from mazegen.errors import MazeValidationError
8 | from mazegen.model import Coordinate, Direction, Maze
9 |
10 |
11 | def shortest_path(
12 |     maze: Maze,
13 |     start: Coordinate | None = None,
14 |     goal: Coordinate | None = None,
15 | ) -> list[Coordinate]:
16 |     """Find a shortest route with breadth-first search.
17 |
18 |     The returned path includes both endpoints. Breadth-first search guarantees
19 |     minimal length because every passage has equal cost.
20 |     """
21 |     origin = maze.entry if start is None else start
22 |     destination = maze.exit if goal is None else goal
23 |     if not maze.in_bounds(origin) or not maze.in_bounds(destination):
24 |         raise MazeValidationError("solution endpoint is outside the maze")
25 |     if origin in maze.blocked or destination in maze.blocked:
26 |         raise MazeValidationError("solution endpoint is a closed cell")
27 |
28 |     pending: deque[Coordinate] = deque([origin])
29 |     previous: dict[Coordinate, Coordinate | None] = {origin: None}
30 |     while pending:
31 |         current = pending.popleft()
32 |         if current == destination:
33 |             break
34 |         for neighbor in maze.open_neighbors(current):
35 |             if neighbor not in previous:
36 |                 previous[neighbor] = current
37 |                 pending.append(neighbor)
38 |
39 |     if destination not in previous:
40 |         raise MazeValidationError(
41 |             f"no route exists from {origin} to {destination}"
42 |         )
43 |     path: list[Coordinate] = []
44 |     cursor: Coordinate | None = destination
45 |     while cursor is not None:
46 |         path.append(cursor)
47 |         cursor = previous[cursor]
48 |     path.reverse()
49 |     return path
50 |
51 |
52 | def path_to_directions(path: list[Coordinate]) -> str:
53 |     """Encode a coordinate path using only N, E, S, and W."""
54 |     direction_for_offset = {
55 |         direction.offset: direction.letter for direction in Direction
56 |     }
57 |     encoded: list[str] = []
58 |     for first, second in zip(path, path[1:]):
59 |         offset = (second[0] - first[0], second[1] - first[1])
60 |         try:
61 |             encoded.append(direction_for_offset[offset])
62 |         except KeyError as exc:
63 |             raise MazeValidationError(
64 |                 "solution path contains non-adjacent cells"
65 |             ) from exc
66 |     return "".join(encoded)

```

10.1 BFS neden shortest path verir?

Her geçidin maliyeti 1'dir. BFS önce entry'ye 1 edge uzaklıktaki, sonra 2 edge uzaklıktaki, sonra 3 edge uzaklıktaki hücreleri işler. Exit ilk kez keşfedildiğinde daha kısa bir yolun işlenmemiş olması mümkün değildir.

`deque.popleft()` O(1) kuyruk işlemi sağlar. `previous` sözlüğü hem visited işlevi görür hem de her hücreye hangi predecessor'dan ulaşıldığını tutar.

10.2 Yolun geri kurulması

- cursor exit'ten başlar.
- previous[cursor] izlenerek entry'ye doğru geriye gidilir.
- Liste ters yönde oluşturduğu için reverse çağrılır.
- Son listede hem entry hem exit bulunur; adım sayısı $\text{len}(\text{path}) - 1$ 'dir.

10.3 NESW dönüşümü

Ardışık iki koordinatın delta'sı hesaplanır. (1,0) E, (-1,0) W, (0,-1) N ve (0,1) S olur. Delta bu dört değerden biri değilse path komşu olmayan hücreler içeriyor demektir ve validation error oluşur.

Örnek yol [(0,0), (1,0), (1,1), (2,1)] tam olarak ESE olur.

11. validate_maze: güvenlik ağı

```

71 | def validate_maze(maze: Maze) -> ValidationReport:
72 |     """Verify wall, connectivity, path, and mode-specific invariants."""
73 |     if len(maze.walls) != maze.height:
74 |         raise MazeValidationError("wall grid height is incorrect")
75 |     if any(len(row) != maze.width for row in maze.walls):
76 |         raise MazeValidationError("wall grid width is incorrect")
77 |     if maze.entry == maze.exit:
78 |         raise MazeValidationError("entry and exit must differ")
79 |     if not maze.in_bounds(maze.entry) or not maze.in_bounds(maze.exit):
80 |         raise MazeValidationError("entry or exit is outside the maze")
81 |     if maze.entry in maze.blocked or maze.exit in maze.blocked:
82 |         raise MazeValidationError("entry or exit is blocked")
83 |
84 |     for y_pos in range(maze.height):
85 |         for x_pos in range(maze.width):
86 |             cell = (x_pos, y_pos)
87 |             value = maze.walls[y_pos][x_pos]
88 |             if not 0 <= value <= ALL_WALLS:
89 |                 raise MazeValidationError(f"invalid wall value at {cell}")
90 |             for neighbor, direction in maze.neighbors(cell):
91 |                 if maze.has_wall(cell, direction) != maze.has_wall(
92 |                     neighbor, direction.opposite
93 |                 ):
94 |                     raise MazeValidationError(
95 |                         f"asymmetric shared wall at {cell}"
96 |                     )
97 |
98 |     for x_pos in range(maze.width):
99 |         if not maze.has_wall((x_pos, 0), Direction.NORTH):
100 |             raise MazeValidationError("north external border is open")
101 |         if not maze.has_wall(
102 |             (x_pos, maze.height - 1), Direction.SOUTH
103 |         ):
104 |             raise MazeValidationError("south external border is open")
105 |     for y_pos in range(maze.height):
106 |         if not maze.has_wall((0, y_pos), Direction.WEST):
107 |             raise MazeValidationError("west external border is open")
108 |         if not maze.has_wall(
109 |             (maze.width - 1, y_pos), Direction.EAST
110 |         ):
111 |             raise MazeValidationError("east external border is open")
112 |
113 |     for x_pos, y_pos in maze.blocked:
114 |         if maze.walls[y_pos][x_pos] != ALL_WALLS:
115 |             raise MazeValidationError("decorative cell is not fully closed")
116 |
117 |     traversable = list(maze.traversable_cells())
118 |     if len(traversable) < 2:
119 |         raise MazeValidationError("maze has fewer than two traversable cells")
120 |     reached: set[Coordinate] = set()
121 |     pending = [traversable[0]]
122 |     while pending:
123 |         current = pending.pop()
124 |         if current in reached:
125 |             continue
126 |         reached.add(current)
127 |         pending.extend(
128 |             neighbor
129 |             for neighbor in maze.open_neighbors(current)
130 |             if neighbor not in reached
131 |         )
132 |     if len(reached) != len(traversable):
133 |         raise MazeValidationError("traversable cells are disconnected")
134 |     degrees = [maze.degree(cell) for cell in traversable]
135 |     if any(degree == 0 for degree in degrees):
136 |         raise MazeValidationError("maze contains an isolated corridor cell")
137 |     if has_open_3x3(maze):
138 |         raise MazeValidationError("maze contains a fully open three-by-three")
139 |
140 |     edges = _count_edges(maze)
141 |     cycle_rank = edges - len(traversable) + 1
142 |     if maze.perfect and cycle_rank != 0:
143 |         raise MazeValidationError("perfect maze is not a tree")
144 |     if not maze.perfect:
145 |         required = {
146 |             (0, 0),
147 |             (maze.width - 1, 0),
148 |             (0, maze.height - 1),
149 |             (maze.width - 1, maze.height - 1),

```

```

150 |         center_cell(maze.width, maze.height),
151 |     }
152 |     if required & maze.blocked:
153 |         raise MazeValidationError("a required corner or center is blocked")
154 |     if cycle_rank < 2:
155 |         raise MazeValidationError(
156 |             "non-perfect maze needs at least two independent cycles"
157 |         )
158 |
159 |     solution = shortest_path(maze)
160 |     return ValidationReport(
161 |         traversable_cells=len(traversable),
162 |         open_edges=edges,
163 |         cycle_rank=cycle_rank,
164 |         dead_ends=sum(degree == 1 for degree in degrees),
165 |         solution_length=len(solution) - 1,
166 |     )
167 |
168 |
169 | def window_is_open(maze: Maze, left: int, top: int) -> bool:
170 |     """Expose the local 3-by-3 check to the generator."""
171 |     return _window_is_open(maze, left, top)

```

11.1 Kontrollerin sırası

Kontrol	Nasıl yapılır	Yakalanan problem
Grid shape	Satır ve sütun uzunlukları	Ragged veya yanlış boyut
Endpoint	Farklı, in-bounds, unblocked	Geçersiz entry/exit
Wall range	$0 \leq \text{value} \leq 15$	Hex dışı bit
Symmetry	direction ve opposite karşılaştır	Tek taraflı duvar açma
Borders	Dört kenarın ilgili bitleri	Maze dışına kaçış
Pattern	blocked hücre değeri F	42 hücresinin açılması
Connectivity	Graph walk ve vertex sayısı	Ulaşılamayan koridor
Isolation	$\text{degree} == 0$ var mı	Tek başına corridor cell
3x3	Bütün windows	Aşırı geniş açık alan
Mode	Cycle rank	Perfect cycle veya zayıf non-perfect
Solution	BFS	Entry-exit kopukluğu

11.2 Edge sayımında double count önleme

Her undirected edge iki hücreden görülebilir. `_count_edges` yalnız EAST ve SOUTH yönlerine bakar. Böylece A-B shared edge'i A'nın EAST'i ve B'nin WEST'i olarak iki kez değil, yalnız bir kez sayılır.

Connectivity doğrulandıktan sonra $\text{cycle rank} = E - V + 1$ güvenle kullanılır. Birden fazla component olsaydı genel formül $E - V + C$ olurdu; burada $C = 1$ validation ile garanti edilmiştir.

11.3 ValidationReport nasıl okunur?

- `traversable_cells`: 42 hücreleri çıkarılmış vertex sayısı.
- `open_edges`: açık shared passage sayısı.
- `cycle_rank`: bağımsız cycle sayısı.
- `dead_ends`: degree 1 olan traversable hücre sayısı.
- `solution_length`: BFS path listesinin edge sayısı.

12. Hexadecimal çıktı dosyası

```

1 | """Required hexadecimal maze output encoding."""
2 |
3 | from __future__ import annotations
4 |
5 | from pathlib import Path
6 |
7 | from mazegen.errors import MazeOutputError
8 | from mazegen.model import Coordinate, Maze
9 | from mazegen.solver import path_to_directions, shortest_path
10 | from mazegen.validation import validate_maze
11 |
12 |
13 | def serialize_maze(
14 |     maze: Maze, solution: list[Coordinate] | None = None
15 | ) -> str:
16 |     """Return the exact grid, coordinate, and shortest-path file content."""
17 |     validate_maze(maze)
18 |     route = shortest_path(maze) if solution is None else solution
19 |     canonical = shortest_path(maze)
20 |     if not route or route[0] != maze.entry or route[-1] != maze.exit:
21 |         raise MazeOutputError("solution must connect entry to exit")
22 |     if len(route) != len(canonical):
23 |         raise MazeOutputError("solution is not a shortest path")
24 |     for first, second in zip(route, route[1:]):
25 |         if second not in maze.open_neighbors(first):
26 |             raise MazeOutputError("solution crosses a closed wall")
27 |     lines = maze.to_hex_lines() + [
28 |         "",
29 |         f"{maze.entry[0]},{maze.entry[1]}",
30 |         f"{maze.exit[0]},{maze.exit[1]}",
31 |         path_to_directions(route),
32 |     ]
33 |     return "\n".join(lines) + "\n"
34 |
35 |
36 | def write_output(
37 |     maze: Maze,
38 |     path: str | Path = "output_maze.txt",
39 |     solution: list[Coordinate] | None = None,
40 | ) -> None:
41 |     """Write a maze to `output_maze.txt` or another selected path."""
42 |     output_path = Path(path)
43 |     content = serialize_maze(maze, solution)
44 |     try:
45 |         with output_path.open("w", encoding="ascii", newline="\n") as stream:
46 |             stream.write(content)
47 |     except PermissionError as exc:
48 |         raise MazeOutputError(
49 |             f"permission denied writing {output_path}"
50 |         ) from exc
51 |     except OSError as exc:
52 |         raise MazeOutputError(f"cannot write {output_path}: {exc}") from exc

```

12.1 serialize_maze neden tekrar doğruluyor?

Maze generate edildikten sonra dış kod walls listesini değiştirebilir. Output sınırında yeniden validate etmek bozuk verinin dosyaya sessizce yazılmasını engeller.

Kullanıcı kendi solution listesini verirse endpoint'ler, canonical BFS uzunluğu ve her adımın open neighbor olması kontrol edilir. Böylece duvardan geçen veya shortest olmayan yol kabul edilmez.

12.2 Exact dosya yapısı

```
<HEIGHT adet hexadecimal satır>
<boş satır>
entry_x,entry_y
exit_x,exit_y
NESW...

# 2x1 exact örnek:
D7

0,0
1,0
E
```

- Her grid karakteri tek hücreyi temsil eder.
- Hex harfleri uppercase A-F kullanılır.
- Grid ile footer arasında tam bir boş satır vardır.
- Footer satırlarında label veya comment yoktur.
- join sonucuna eklenen son '\n', final newline garantisidir.
- Dosya ascii encoding ve newline='\n' ile yazılır.

13. Hata sınıfları ve hata sınırları

```

1 | """Exceptions raised by the maze algorithm package."""
2 |
3 |
4 | class MazeError(Exception):
5 |     """Base class for expected maze errors."""
6 |
7 |
8 | class MazeGenerationError(MazeError):
9 |     """Raised when the requested maze cannot be generated."""
10 |
11 |
12 | class MazeValidationError(MazeError):
13 |     """Raised when a maze violates a structural invariant."""
14 |
15 |
16 | class MazeOutputError(MazeError):
17 |     """Raised when maze output cannot be encoded or written."""

```

Hata	Nerede	Örnek
MazeGenerationError	Parametre veya generation	Out-of-bounds entry, imkansız cycle kapasitesi
MazeValidationError	Solver ve validator	Disconnected maze, asymmetric wall, route yok
MazeOutputError	Serializer ve writer	Shortest olmayan path, permission denied
ValueError	Düşük seviye model API	External border açmaya çalışma

MazeError ortak base class'tır. Üst katman isterse yalnız MazeError yakalar, isterse generation/output ayırımına göre daha spesifik mesaj verir. Bu modüller traceback basmaz; exception üretir. Kullanıcıya mesaj basmak CLI katmanının sorumluluğudur.

write_output yalnız expected filesystem hatalarını çevirir. Programlama hatalarını broad except ile saklamaz.

14. Testleri okuyarak tasarımı öğrenmek

```

25 | def generated_maze(
26 |     *, perfect: bool = True, seed: int = 42,
27 |     width: int = 20, height: int = 15
28 | ) -> Maze:
29 |     """Return a standard generated maze for a test."""
30 |     return MazeGenerator(
31 |         width=width,
32 |         height=height,
33 |         entry=(0, 0),
34 |         exit=(width - 1, height - 1),
35 |         seed=seed,
36 |         perfect=perfect,
37 |     ).generate()
38 |
39 |
40 | class MazeAlgorithmTests(unittest.TestCase):
41 |     """Exercise all algorithm responsibilities assigned to this module."""
42 |
43 |     def test_seed_is_deterministic(self) -> None:
44 |         """Equal seeds create equal wall grids."""
45 |         self.assertEqual(
46 |             generated_maze(seed=123).walls,
47 |             generated_maze(seed=123).walls,
48 |         )
49 |         self.assertNotEqual(
50 |             generated_maze(seed=123).walls,
51 |             generated_maze(seed=124).walls,
52 |         )
53 |
54 |     def test_wall_symmetry_and_closed_borders(self) -> None:
55 |         """Shared walls agree and no external border is opened."""
56 |         maze = generated_maze(perfect=False)
57 |         for cell in maze.traversable_cells():
58 |             for neighbor, direction in maze.neighbors(cell):
59 |                 self.assertEqual(
60 |                     maze.has_wall(cell, direction),
61 |                     maze.has_wall(neighbor, direction.opposite),
62 |                 )
63 |             self.assertTrue(
64 |                 all(
65 |                     maze.has_wall((x_pos, 0), Direction.NORTH)
66 |                     for x_pos in range(maze.width)
67 |                 )
68 |             )
69 |             self.assertTrue(
70 |                 all(
71 |                     maze.has_wall(
72 |                         (x_pos, maze.height - 1), Direction.SOUTH
73 |                     )
74 |                     for x_pos in range(maze.width)
75 |                 )
76 |             )
77 |
78 |     def test_perfect_maze_is_connected_tree(self) -> None:
79 |         """Perfect mode has zero cycles and exactly V-1 edges."""
80 |         report = validate_maze(generated_maze(perfect=True))
81 |         self.assertEqual(report.cycle_rank, 0)
82 |         self.assertEqual(report.open_edges, report.traversable_cells - 1)
83 |
84 |     def test_non_perfect_maze_has_many_routes(self) -> None:
85 |         """Non-perfect mode has at least two independent cycles."""
86 |         report = validate_maze(generated_maze(perfect=False))
87 |         self.assertGreaterEqual(report.cycle_rank, 2)
88 |
89 |     def test_no_open_three_by_three_area(self) -> None:
90 |         """Neither mode creates a fully open 3-by-3 region."""
91 |         for perfect in (True, False):
92 |             for seed in range(10):
93 |                 with self.subTest(perfect=perfect, seed=seed):
94 |                     self.assertFalse(
95 |                         has_open_3x3(
96 |                             generated_maze(perfect=perfect, seed=seed)
97 |                         )
98 |                     )
99 |
100 |     def test_pattern_cells_are_f_and_not_route_cells(self) -> None:
101 |         """All decorative cells are fully closed and avoided by the solver."""
102 |         maze = generated_maze()
103 |         route = set(shortest_path(maze))

```

```

104 |         self.assertEqual(len(maze.blocked), 20)
105 |         self.assertFalse(route & maze.blocked)
106 |         for x_pos, y_pos in maze.blocked:
107 |             self.assertEqual(maze.walls[y_pos][x_pos], ALL_WALLS)
108 |
109 |     def test_small_rectangular_maze_without_pattern(self) -> None:
110 |         """Small rectangular mazes remain valid when 42 cannot fit."""
111 |         generator = MazeGenerator(4, 7, (0, 0), (3, 6), seed=8)
112 |         maze = generator.generate()
113 |         self.assertFalse(maze.blocked)
114 |         self.assertTrue(generator.diagnostics)
115 |         validate_maze(maze)
116 |
117 |     def test_shortest_path_follows_open_walls(self) -> None:
118 |         """The BFS solution connects the endpoints without crossing walls."""
119 |         maze = generated_maze(perfect=False, seed=99)
120 |         route = shortest_path(maze)
121 |         self.assertEqual(route[0], maze.entry)
122 |         self.assertEqual(route[-1], maze.exit)
123 |         for first, second in zip(route, route[1:]):
124 |             self.assertIn(second, maze.open_neighbors(first))
125 |         distances: dict[tuple[int, int], int] = {maze.entry: 0}
126 |         pending: deque[tuple[int, int]] = deque([maze.entry])
127 |         while pending:
128 |             current = pending.popleft()
129 |             for neighbor in maze.open_neighbors(current):
130 |                 if neighbor not in distances:
131 |                     distances[neighbor] = distances[current] + 1
132 |                     pending.append(neighbor)
133 |         self.assertEqual(len(route) - 1, distances[maze.exit])
134 |
135 |     def test_invalid_requests_are_rejected(self) -> None:
136 |         """Bad dimensions and coordinates raise generation errors."""
137 |         invalid = (
138 |             (0, 5, (0, 0), (0, 1), True),
139 |             (5, 5, (-1, 0), (4, 4), True),
140 |             (5, 5, (0, 0), (5, 4), True),
141 |             (5, 5, (2, 2), (2, 2), True),
142 |             (2, 2, (0, 0), (1, 1), False),
143 |         )
144 |         for width, height, entry, exit_cell, perfect in invalid:
145 |             with self.subTest(width=width, height=height):
146 |                 with self.assertRaises(MazeGenerationError):
147 |                     MazeGenerator(
148 |                         width,
149 |                         height,
150 |                         entry,
151 |                         exit_cell,
152 |                         perfect=perfect,
153 |                     )
154 |
155 |     def test_exact_hexadecimal_output(self) -> None:
156 |         """Output has uppercase cells, footer, solution, and final newline."""
157 |         maze = generated_maze(width=10, height=8)
158 |         content = serialize_maze(maze)
159 |         lines = content.splitlines()
160 |         self.assertTrue(content.endswith("\n"))
161 |         self.assertEqual(len(lines), maze.height + 4)
162 |         for row in lines[:maze.height]:
163 |             self.assertEqual(len(row), maze.width)
164 |             self.assertIsNotNone(re.fullmatch(r"[0-9A-F]+", row))
165 |         self.assertEqual(lines[maze.height], "")
166 |         self.assertEqual(lines[maze.height + 1], "0,0")
167 |         self.assertEqual(lines[maze.height + 2], "9,7")
168 |         self.assertIsNotNone(
169 |             re.fullmatch(r"[NESW]+", lines[maze.height + 3])
170 |         )
171 |
172 |     def test_known_masks_and_file_writer(self) -> None:
173 |         """One horizontal passage encodes as D7 and writes exact bytes."""
174 |         maze = generated_maze(width=2, height=1, seed=5)
175 |         expected = "D7\n\n0,0\n1,0\nE\n"
176 |         self.assertEqual(serialize_maze(maze), expected)
177 |         with tempfile.TemporaryDirectory() as directory:
178 |             output_path = Path(directory) / "output_maze.txt"
179 |             write_output(maze, output_path)
180 |             self.assertEqual(
181 |                 output_path.read_text(encoding="ascii"), expected
182 |             )
183 |
184 |
185 | if __name__ == "__main__":
186 |     unittest.main()

```

14.1 Her test neyi kanıtlıyor?

Test	Koruduğu sözleşme
seed_is_deterministic	Aynı seed aynı grid, farklı seed normalde farklı grid
wall_symmetry...	Shared wall coherence ve external border
perfect...tree	Connected tree: rank 0 ve $E=V-1$
non_perfect...routes	Cycle rank en az 2
no_open_three...	Her iki modda 3x3 yasağı
pattern_cells...	20 adet F hücresi ve route ile kesişmeme
small_rectangular...	Pattern olmadan devam ve rectangular destek
shortest_path...	Open-edge adımları ve bağımsız BFS mesafesi
invalid_requests...	Hatalı geometry'nin erken reddi
exact_hexadecimal...	Satır sayısı, charset, footer, final newline
known_masks...	D7 bit örneği ve gerçek file write

Analyzer ile çapraz doğrulama

Repository'deki maze_analyzer.py üreticiden bağımsız parse ve graph analizi yapar. output_maze.txt için wall coherence OK, 0 unreachable corridor ve PERFECT verdict; non-perfect örnek için 27 loop, 0 ordinary dead-end ve Pac-Man-USABLE verdict alınmıştır.

15. Karmaşıklık ve kaynak kullanımı

İşlem	Zaman	Ek bellek	Not
DFS tree	$O(V + E)$	$O(V)$	reached + stack
BFS shortest path	$O(V + E)$	$O(V)$	queue + previous
Final validation	$O(V + E)$	$O(V)$	connectivity ve windows
Hex serialization	$O(V + P)$	$O(V + P)$	P = solution adımı
Pattern candidate	Aday başına $O(V+E)$	$O(V)$	Merkeze yakın güvenli aday genelde erken bulunur
Braiding	Yaklaşık $O(V+E)$	$O(E)$	3x3 kontrolü local sabit pencere

Dikdörtgen grid için potential E en fazla $2WH - W - H$ 'dir. Bu nedenle E, V ile aynı büyüklük mertebesinde ve traversal'lar pratikte lineerdir.

MAX_CELLS = 250000 koruması hem walls matrisinin hem set/dict tabanlı graph işlemlerinin kontrolsüz büyümesini önler.

16. Uçtan uca kullanım ve entegrasyon

```
from mazegen import MazeGenerator, write_output

generator = MazeGenerator(
    width=20,
    height=15,
    entry=(0, 0),
    exit=(19, 14),
    seed=42,
    perfect=False,
)
maze = generator.generate()
solution = generator.shortest_path()
write_output(maze, 'output_maze.txt', solution)

print(generator.report)
print(maze.walls[0][0])
print(len(solution) - 1)
```

16.1 Teammate katmanı ne yapmalı?

- Config parser WIDTH/HEIGHT/ENTRY/EXIT/PERFECT/SEED değerlerini üretir.
- Bu değerlerle MazeGenerator instantiate edilir.
- generate sonucu UI'ya verilir; UI walls üzerinde yalnız read yapar.
- write_output configured path ile çağrılır.
- MazeError üst katmanda yakalanıp kullanıcıya traceback'siz mesaj basılır.
- Regeneration için yeni bir seed seçilir veya mevcut seed artırılır.

16.2 Modül sınırını bozmama

Generator input() çağırmmalı, terminal rengi bilmemeli ve config file parse etmemelidir. Solver output file yazmamalıdır. Model BFS bilmemelidir. Bu ayırım birim testlerini ve ileride algoritma değiştirmeyi kolaylaştırır.

UI solution göstermek istediğinde shortest_path(maze) çağırabilir; maze grid'i içine '*' gibi display karakterleri yazmamalıdır. Hex wall değeri yalnız topolojiyi temsil etmeye devam eder.

17. Debug rehberi: hata nerede aranır?

Belirti	Olası neden	Kontrol
Analyzer INCOHERENT	Duvar tek taraftan değişti	Her zaman remove_wall/add_wall kullan
Entry exit unreachable	Blocked yerleşimi veya carve eksik	reached count ve pattern connectivity
Perfect ama loop var	Visited komşuya da edge açıldı	Candidate filter item[0] not in reached
Non-perfect rank 1	Yalnız bir extra wall açıldı	opened ve target_rank
3x3 validation hatası	Candidate local check atlandı	Bütün extra openings _try_open üzerinden mi?
Pattern hücresi F değil	Blocked hücreye passage açıldı	remove_wall blocked guard
Footer path yanlış	x/y veya delta karıştı	walls[y][x], Coordinate=(x,y)
Seed değişmeden sonuç değişiyor	Global RNG veya unordered random seçim	Local Random ve stable base order

17.1 İnceleme sırasında faydalı ifadeler

```

report = validate_maze(maze)
print(report)
print('seed:', maze.seed)
print('blocked:', len(maze.blocked))
print('entry degree:', maze.degree(maze.entry))
print('exit degree:', maze.degree(maze.exit))
print('hex first row:', maze.to_hex_lines()[0])
print('path length:', len(shortest_path(maze)) - 1)

```

18. Savunma soruları ve kısa cevaplar

S: Neden DFS seçtin?

C: Basit, iteratif uygulanabilir, seed ile deterministik ve spanning tree garantisi doğrudan.

S: Perfect olduğunu path bularak mı kanıtlıyorsun?

C: Hayır. Connectivity ve $E=V-1$ /cycle rank 0 ile yapısal olarak kanıtlıyorum.

S: Neden BFS?

C: Tüm edge maliyetleri 1; BFS katman sırasıyla ilerlediği için shortest path garantili.

S: Duvar coherence nasıl korunuyor?

C: remove_wall ve add_wall yönün opposite bitini komşuda aynı anda değiştiriyor.

S: F ne demek?

C: 1111; NORTH+EAST+SOUTH+WEST, yani dört duvar da kapalı.

S: 42 bağlantıyı keserse ne olur?

C: Placement öncesi complement connectivity test edilir; güvenli aday yoksa pattern atlanır.

S: Non-perfect neden gerçekten non-perfect?

C: Cycle rank validator tarafından en az 2 olarak zorunlu tutulur.

S: Dead-end nasıl azaltılıyor?

C: Degree 1 hücreler bulunup güvenli kapalı komşu duvarları açılır; üç pass uygulanır.

S: 3x3 nasıl korunuyor?

C: Her extra opening geçici uygulanır, etkilenen local 3x3 windows taranır, sonra karar verilir.

S: Neden borders kapalı kalır?

C: Model external neighbor'a remove_wall izni vermez; generation yalnız in-bounds shared edges kullanır.

S: Neden output öncesi tekrar validation?

C: Dış kodun mutable walls listesini sonradan bozmuş olabileceği varsayılır.

S: x ve y nerede kullanılıyor?

C: Coordinate=(x,y), matris erişimi walls[y][x].

S: Bir edge neden iki kere sayılmıyor?

C: Edge count yalnız EAST ve SOUTH yönlerinde yapılır.

S: Seed None olursa?

C: 64-bit random seed seçilir ve generator.seed üzerinden yeniden üretilebilir.

19. Kendini sına

19.1 Kod okumadan cevapla

- A = 1010 hücresinde hangi duvarlar kapalıdır?
- Bir duvarı yalnız current hücrede açarsan validator hangi kontrolü düşürür?
- V=280 ve E=279 connected graph neden perfect'tir?
- V=280, E=281 için cycle rank kaçır?
- DFS candidate listesine reached hücreleri dahil edersen ne bozulur?
- BFS previous sözlüğü iki ayrı görevi nasıl yapar?
- Neden blocked hücreler dead-end sayımına dahil değildir?
- 2x1 horizontal passage neden D7 olarak çıkar?
- Non-perfect mode'da bir extra edge neden yeterli değildir?
- Bir path'in shortest olduğu output sınırında nasıl kontrol edilir?

19.2 Cevap anahtarı

- A = 8 + 2: EAST ve WEST kapalı, NORTH ve SOUTH açık.
- Shared-wall symmetry/coherence kontrolü.
- E=V-1 ve connected olduğu için tree; cycle rank 0.
- $281 - 280 + 1 = 2$.
- Eski vertex'e edge açıp cycle oluşturabilir; perfect garantisi kaybolur.
- Hem visited set işlevi görür hem path reconstruction predecessor bilgisini tutar.
- Onlar corridor vertex değildir; dekoratif tamamen kapalı engellerdir.
- Solda EAST açık olduğundan $N+S+W=D$; sağda WEST açık olduğundan $N+E+S=7$.
- Rank 1 yalnız tek bağımsız loop'tur; şart rank ≥ 2 'dir.
- Endpoint, open-edge adımları ve canonical BFS ile aynı uzunluk kontrol edilir.

20. Son tekrar: ezberlenmesi gereken çekirdek

Duvar formülü

$N=1$, $E=2$, $S=4$, $W=8$; bit 1 ise kapalı; F tamamen kapalı.

Perfect formülü

Connected + $E=V-1 \rightarrow$ tree $\rightarrow$ cycle rank 0 $\rightarrow$ her çift arasında tek yol.

Non-perfect formülü

Tree + güvenli extra edges; cycle rank = $E-V+1 \geq 2$; dead-end braiding.

Shortest path

BFS katman sırası + previous map + reverse; output NESW.

En kritik invariant

Bir shared wall iki hücrede aynı anda değiştirilir. Dış sınırlar hiç açılmaz.

Bu beş maddeyi kod satırlarıyla ilişkilendirerek açıklayabiliyorsan algoritma bölümünü yalnız çalıştırmıyor, gerçekten savunabiliyorsun.